CMPE 172 — Milestone 3

Integration, Distribution & Deployment

Project: Nail Salon Appointment Scheduler

Student: FirstName LastName

Verification date: October 27, 2026

Repository branch: milestone3

1. Project overview

This milestone extends the existing Java 21 Spring Boot 4.1.1 application. It preserves JdbcTemplate repositories, PostgreSQL, Thymeleaf/Bootstrap pages, session authentication and customer/provider authorization. Existing registration, booking, cancellation fees, provider availability, history and double-booking protection remain in place. The original slot row lock and partial unique index still protect bookings.

The implementation adds an HTTP notification boundary, commit-aware coordination, receiver idempotency, operational endpoints, Docker deployment, CI and automated verification. No real email, SMS provider, broker or paid service is used.

2. Distribution boundary and coarse-grained interface

The client uses either `POST /customer/appointments` and `DELETE /customer/appointments/{id}`, or the website's `/web/book` and cancellation form. Both controller paths call the same `AppointmentService` methods. Controllers do not implement delivery logic.

`NotificationGateway.send(NotificationRequest)` makes a real HTTP `POST /notifications`. `NotificationController` is the second REST controller in the existing project. By default it runs in the same application process, but the gateway URL can point at another instance. This demonstrates a logical HTTP distribution boundary; the default Compose setup has two containers (scheduler and PostgreSQL), not a separately deployed notification microservice.

The DTO carries one complete business notification:

```
public record NotificationRequest(
    @NotBlank @Size(max = 100)
    @Pattern(regexp = &amp;quot;[A-Za-z0-9_-]+&amp;quot;) String notificationId,
    @NotBlank @Size(max = 100) String recipient,
    @NotBlank
    @Pattern(regexp = &amp;quot;BOOKING_CONFIRMATION|CANCELLATION_CONFIRMATION&amp;quot;) String type,
    @NotBlank @Size(max = 1000) String message) {}
```

A single request contains the routing identifier, business type and full message. The receiver does not need callbacks to look up appointment or customer data. This reduces HTTP chatter and coupling. The new ID is required; the initial three-field prototype's callers must supply it. `@Valid` rejects malformed, missing, blank, unsupported and oversized fields with HTTP 400.

Recipients use opaque internal identifiers such as `customer:42`. Messages contain the appointment ID and start time; booking messages also identify the slot, while cancellation messages include the calculated fee. The mock logs only the notification ID and type. It stores a payload fingerprint, not recipient or message text, and

sends no external communications.

3. Commit-then-notify transaction design

`AppointmentService.bookAppointment` retains its existing `@Transactional(isolation = READ_COMMITTED)` boundary. It locks the availability row, validates the time and active booking state, inserts the appointment, and publishes an immutable snapshot. Cancellation locks the owned appointment, validates its status/time, computes the existing fee, updates the row and publishes its snapshot.

```
Long id = appointmentRepository.createAppointment(customerId, slotId);
events.publishEvent(new AppointmentNotification(new NotificationRequest(
    &amp;quot;booking-&amp;quot; + id, &amp;quot;customer:&amp;quot; + customerId, &amp;quot;BOOKING_CONFIRMATION&amp;quot;,
    &amp;quot;Appointment #&amp;quot; + id + &amp;quot; booked for slot #&amp;quot; + slotId
        + &amp;quot; starting &amp;quot; + startAt + &amp;quot;.&amp;quot;)));
return id;
```

Publishing an event does not send HTTP. `AppointmentNotificationListener.afterCommit` is registered for successful transaction completion:

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void afterCommit(AppointmentNotification event) {
    // Increment committed-operation metric, then call gateway.send(...).
}
```

`AFTER_COMMIT` suppresses notifications when the transaction rolls back, including a larger caller transaction that rolls back after `bookAppointment` returns. Spring can retain bound resources during completion callbacks; `NOT_SUPPORTED` suspends those resources before the listener calls the gateway. The gateway also rejects an active transaction as a defensive invariant. The listener catches unexpected runtime delivery errors, so they cannot change an already committed appointment response.

Delivery runs synchronously after commit, so retries add response latency even though database row locks have been released. This keeps the project understandable without introducing a queue, asynchronous executor or broker.

4. Timeouts, retries, fallback and idempotency

`NotificationProperties` exposes validated settings under `salon.notification`. Environment variables override the URL, connection timeout (default 500 ms), response/read timeout (1000 ms), maximum attempts (3) and fixed retry delay (100 ms). Attempts are limited to 1–5; timeout and delay values also have upper bounds. The URL must be absolute HTTP(S), without credentials, query or fragment.

The gateway configures a `SimpleClientHttpRequestFactory` explicitly rather than relying on classpath defaults. It retries transport failures and HTTP 408, 429, 500, 502, 503 and 504. Other HTTP errors stop immediately. An interruption restores the thread's interrupt status and stops retries. Exhaustion returns `false`, emits a WARN fallback event and retains the committed booking/cancellation.

`booking-{appointmentId}` and `cancellation-{appointmentId}` remain unchanged across attempts. `NotificationInbox.accept` runs in its own receiver transaction and inserts a receipt with `ON CONFLICT DO NOTHING`. A unique PostgreSQL primary key handles simultaneous retries and multiple application instances. Matching payload hashes return HTTP 200 `DUPLICATE`; reusing an ID with different content returns HTTP 409. New receipts return HTTP 200 `RECEIVED`. Length-prefixed fields feed SHA-256 so field boundaries are unambiguous. Receipts persist across process restarts and are not automatically expired.

These mechanisms provide bounded best-effort delivery and deduplicated mock acceptance. They do not guarantee delivery. A crash after appointment commit but before HTTP, or a prolonged outage beyond the retry budget, can lose the notification. There is no durable sender outbox, background replay or exactly-once external delivery. A crash between receipt commit and the mock INFO log can also omit that log; the receipt remains recorded. A future durable outbox/worker would be appropriate if guaranteed eventual delivery became a requirement.

5. Structured logging

SLF4J messages use stable `event=` names and key/value fields suitable for searching. These are actual statements in the implementation:

```
// AppointmentNotificationListener: successful committed operation, INFO
log.info(&amp;quot;event=appointment_committed notificationId={} type={}&amp;quot;,
        request.notificationId(), request.type());

// NotificationGateway: retry and fallback, WARN
log.warn(&amp;quot;event=notification_retry notificationId={} nextAttempt={}&amp;quot;,
        notification.notificationId(), attempt + 1);
log.warn(&amp;quot;event=notification_fallback notificationId={} type={} action=retain_appointment&amp;quot;,
        notification.notificationId(), notification.type());

// GlobalExceptionHandler: unexpected request/database failures, ERROR
log.error(&amp;quot;event=request_unexpected_failure path={} cause={}&amp;quot;,
        request.getRequestURI(), e.getClass().getSimpleName());
```

`HealthController` logs database-check failures at ERROR, and the listener logs unexpected delivery exceptions at ERROR. Transport failures log a safe exception class at WARN. Application messages avoid passwords, customer names, recipient addresses, message bodies, SQL text and raw exception messages. Standard framework diagnostic logging should remain at INFO in deployment.

6. Health and database connectivity

Public `GET /health` calls PostgreSQL using `JdbcTemplate.queryForObject("SELECT 1", Integer.class)`. A successful query returns HTTP 200:

```
{&amp;quot;status&amp;quot;:&amp;quot;UP&amp;quot;,&amp;quot;application&amp;quot;:&amp;quot;UP&amp;quot;,&amp;quot;database&amp;quot;:&amp;quot;UP&amp;quot;}
```

An unavailable or unsuccessful database check returns HTTP 503 with overall/database `DOWN` and application `UP`. This describes a running HTTP process that cannot use its database. If the process itself is stopped, the caller receives a connection failure instead of JSON. The response omits database URLs, credentials and exception details.

Hikari connection acquisition/validation limits and PostgreSQL connect/socket limits bound common database failure paths; the JDBC template also has a query timeout. The Docker application health check uses this endpoint, while PostgreSQL's own readiness check uses `pg_isready`. If the database is unavailable at initial startup, the existing mandatory schema initialization may prevent the application from starting; Compose waits for database readiness first.

7. Application metrics

Spring Boot Actuator supplies Micrometer and `/actuator/metrics`. Only `health` and `metrics` are exposed. Counters are registered at startup and incremented in the after-commit listener:

- `salon.bookings.success`: successfully committed bookings.
- `salon.cancellations.success`: successfully committed cancellations.
- `salon.notifications.fallback`: gateway calls that exhaust delivery or stop on permanent failure.

Inspect with `curl http://localhost:8080/actuator/metrics/salon.bookings.success`. A rollback does not increment the booking counter, and retry attempts do not inflate it. Counters are process-local and reset on restart; they are not durable accounting records. The fallback counter tracks ordinary gateway `false` outcomes; unexpected listener errors have a separate ERROR log. Local demonstration endpoints are public; an Internet deployment should restrict management and mock receiver access at its network boundary.

8. Failure and recovery scenario

To simulate receiver unavailability in Compose, run `NOTIFICATION_BASE_URL=http://127.0.0.1:1 docker compose up -d --force-recreate app`. Book and cancel through the website. Each operation still succeeds and persists while transport, retry and fallback WARN events appear. Verify history and the fallback counter. Port 1 is deliberately unavailable in this demonstration.

Restore `NOTIFICATION_BASE_URL=http://app:8080 docker compose up -d --force-recreate app`. Subsequent operations deliver normally. Historical failed notifications are not automatically replayed. If an operator chooses to replay one, they must reconstruct and retain the original ID and exact payload; receipt deduplication protects against uncertain acknowledgments. Availability of the database and appointment history remains authoritative.

9. Docker deployment

The multi-stage `Dockerfile` builds the existing Maven project using a Java 21 JDK and runs the executable JAR using a Java 21 JRE under a non-root user. The runtime includes curl for the health probe. The Docker build skips test execution; CI and local `verify` run tests separately. `.dockerignore` admits only the Maven wrapper, POM and source tree, keeping local credentials and generated artifacts out of the build context.

`docker-compose.yml` starts PostgreSQL 16 with a named persistent volume and the Spring application after the database becomes healthy. JDBC connects to `db:5432`, never host localhost. The mock URL defaults to `http://app:8080`. Database credentials come from environment variables; `.env.example` contains a placeholder and `.env` is ignored. The host HTTP port binds to loopback. Existing `schema.sql` and `seed.sql` still run on application startup, including the additive receipt table. No database initialization behavior was replaced.

```
cp .env.example .env
# Replace the password placeholder locally before starting.
docker compose up --build
# Stop containers while preserving the database volume:
docker compose down
```

Compose syntax validation passed. Image build, startup and container health behavior were not executed locally because the Docker daemon was unavailable. Changing the environment password does not alter credentials already stored in a PostgreSQL volume; rotate the database role explicitly when retaining existing data.

10. GitHub Actions CI

`.github/workflows/ci.yml` runs on every push and pull request. It checks out source, installs Temurin Java 21, uses Maven caching and runs `./mvnw -B clean verify`. A healthy PostgreSQL 16 service provides the dedicated `salon_m2_test` database. The disposable CI service uses trust authentication only for that job, so there are no committed database passwords or required local files. This trust configuration is not used by deployment Compose.

A follow-up assertion requires the local PostgreSQL workflow suite to have at least 16 tests with no skips, errors or failures. This prevents a green unit-only build from hiding skipped integration tests. Testcontainers can also run the inherited suite when Docker is available. Surefire results upload even on failure. No hosted Actions run is claimed: this implementation was not pushed.

11. Automated tests and verified results

The October 27, 2026 local `clean verify` completed with **BUILD SUCCESS: 75 discovered, 59 passed, 0 failures, 0 errors, 16 skipped**. The 16 skipped cases belong exclusively to the Docker-backed duplicate workflow suite. All 16 cases in `LocalPostgresTest` ran against temporary PostgreSQL 14.19 (Homebrew). Compose and CI target PostgreSQL 16; that version was not exercised locally. The local JDK was 22.0.2, compiling with release 21; Java 21 runtime execution remains for CI/Docker verification.

`NotificationControllerTest` checks acceptance, duplicate acknowledgments and malformed/invalid inputs. `NotificationGatewayTest` uses its own ephemeral JDK HTTP server for JSON delivery, transient failures, stable retry payloads, recovery, permanent errors and read timeouts; it also checks connection-refused fallback and the active-transaction guard. `HealthControllerTest` verifies UP/200 and DOWN/503 without exposing failure details. A listener test verifies unexpected gateway errors do not escape.

The critical integration method is `committedBookingAndCancellationSurviveUnavailableNotificationService` in `PostgresWorkflowSuite`. It calls the real Spring transactional service through the REST controller and the real gateway with an unavailable HTTP URL. Before each HTTP attempt, an independent database connection observes BOOKED or CANCELLED, and the test confirms there is no active sending transaction. The HTTP response remains successful and both real delivery results are fallback. Assertions after the callback verify these observations so transaction callback exception handling cannot hide missing evidence.

Additional database cases cover website booking/cancellation during outage, failed booking and outer rollback without notifications or metric increments, cancellation rollback, sequential/concurrent receiver deduplication, changed-payload conflicts, public database health and exposed metrics. The existing synchronized two-customer booking test and database unique-index backstop still pass, along with the Milestone 2 authentication, provider, registration, cancellation fee and page tests.

The packaged JAR also passed **58 curl HTTP smoke checks**, including actual HTTP receipt of booking and cancellation notifications. A live PostgreSQL stop/restart changed `/health` from 200/UP to 503/DOWN and back to 200/UP. These checks used only temporary databases.

See `Milestone3_Test_Results.json` for per-suite counts and `Milestone3_Verification.md` for the complete verification scope. Raw reports regenerate under `target/surefire-reports/`. Docker execution and hosted CI outcomes remain unverified.

12. Distribution diagram and code walkthrough

`Milestone3_Distribution.png`, `.pdf` and `.mmd` show the client, shared scheduler, database transaction, after-commit listener, gateway, HTTP receiver, durable receipt table and timeout/retry/fallback path. `Milestone3_Demo_Script.md` provides a 5–7 minute source-focused walkthrough and runnable demonstration commands. No video recording or submission link is claimed.

13. Deliverables and limitations

The submission ZIP includes the updated source/tests, Maven wrapper and POM, Docker/CI configuration, README, report in Markdown/PDF, diagram, verification summaries and demo script. It excludes `.git`, local `.env`, dependency caches, runtime cookie jars, database files and compiled outputs. `FirstName LastName` and the ZIP filename intentionally retain placeholders for the student's identity.

Remaining operational limitations are best-effort sender delivery without an outbox, synchronous retry latency, process-local metrics, persistent receipt retention without a purge policy, and unverified Docker/Java 21 runtime/hosted CI execution. All authorized implementation work and locally available checks are represented accurately; no real notifications are sent.

References

• Spring transaction-bound events: https://docs.spring.io/spring-framework/reference/data-access/transaction/event.html

• Spring `TransactionalEventListener` resource lifecycle: https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/event/TransactionalEventListener.html

• Spring explicit HTTP client timeouts: https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/client/SimpleClientHttpRequestFactory.html

Distribution diagram

